Taskorbit Conversational Agent System Architecture

1. Runtime Components Overview

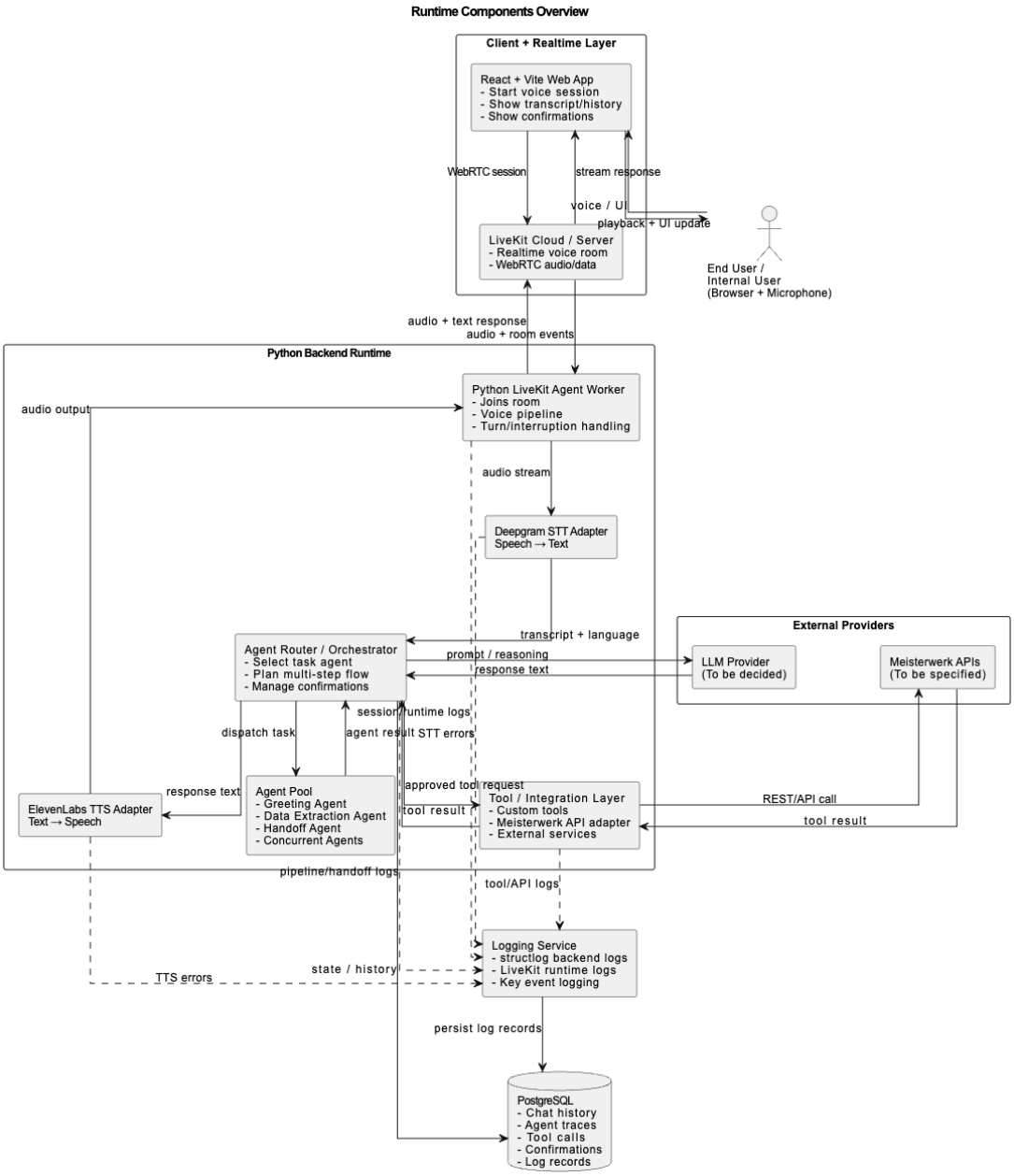


Figure 1. Runtime architecture overview of the planned voice agent system.

2. Code Components Overview

Purpose: Show the initial logical structure of the codebase and major dependency directions.

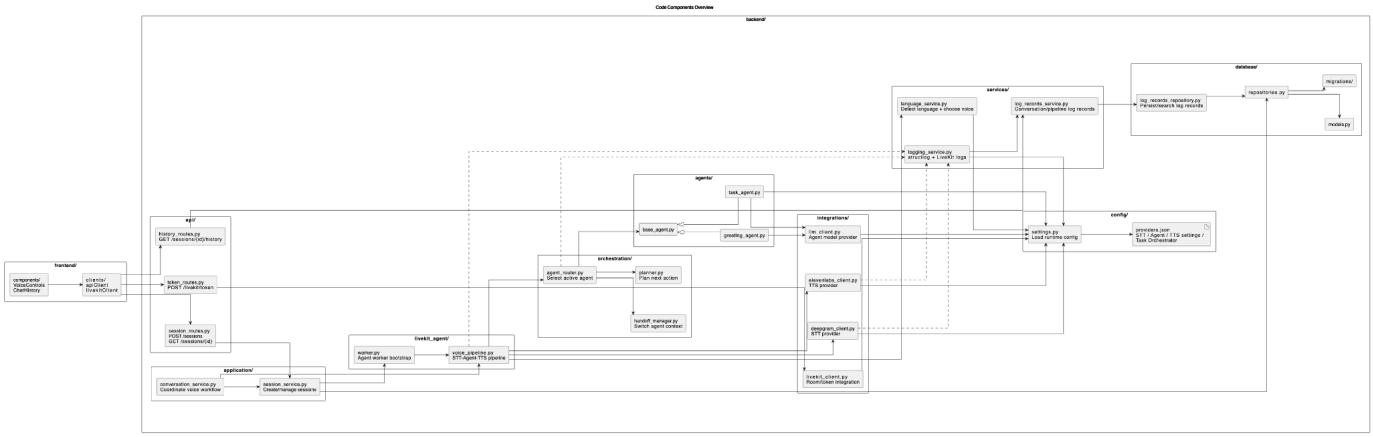


Figure 2. Logical code architecture and main dependencies.

3. Technology Stack Summary

Area	Technology	Role
Frontend	**React.js**	**Web UI and LiveKit client to maintain a conversation between the user and the agent**
Backend	**Python**	**Main orchestration layer for voice pipeline, agent logic, extraction workflows, LLM integration, API calls, and pipeline control**
Voice Streaming/Audio Processing	**LiveKit + LiveKit Agents**	**Real-time audio communication between user and the agent** **Manages rooms, real-time audio transport, and agent-related media workflows** **An agent worker joins LiveKit rooms and runs the voice pipeline with STT, LLM, TTS, interruption/turn handling, and session control.**
Orchestration	**JSON task definitions**	**Orchestration engine dynamically selects and executes task-specific prompts (Task switching)**
Agent/LLM	**Open AI model or Gemini**	**Reasoning and response generation**

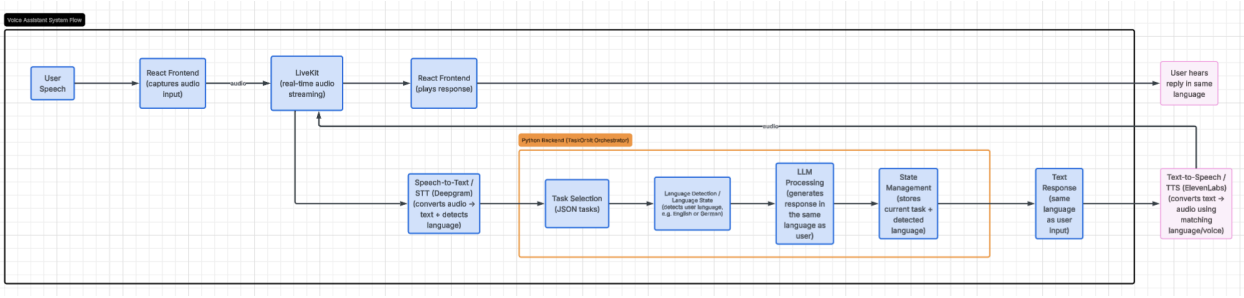
	(to be decided by stakeholders)	
STT (Speech to Text)	**Deepgram**	**Speech-to-text provider for converting user voice input into transcripts**
TTS	**ElevenLabs**	**Converts generated responses back to spoken output in the user's language.**
Database	**PostgreSQL**	**Persistent storage for conversations, messages, agent traces, tool calls, confirmations, and execution history**
Cloud	**Google Cloud Platform**	**Deployment, scaling, database access**
Logging/Monitoring	**Pytest, Ruff/Black, Structured logging, Cypress**	**Debugging and reliability tracking Testing, formatting, linting, and consistent logging for maintainability.**
Containerization	**Docker**	**Create a portable for each unit that can run on any system and its relevant service.**

4. Textual Explanation and Key Decisions

4.1 Architecture Overview

Speech-to-Text (STT) → Agent (Orchestration Engine) → Text-to-Speech (TTS)

- The system is a real-time, voice-enabled AI platform where users interact through a web application connected via LiveKit for low-latency audio streaming and communication.
- User audio is captured via the React frontend and streamed using LiveKit. This text is then processed by the backend agent, implemented in Python, which generates a response using an LLM.
- User speech is processed in a Python backend runtime, where:
    - Audio is converted to text using a Speech-to-Text (STT) service (using Deepgram)
    - A central Agent Router / Orchestrator interprets the input using an LLM, plans multi-step workflows, and manages confirmations and control flow
- The orchestrator delegates tasks to a pool of specialized agents (e.g., greeting, data extraction, handoff), enabling modular and concurrent task execution
- For external actions, the system uses a Tool / Integration Layer to call APIs (e.g., Meisterwerk services) and other external systems, returning structured results back to the orchestrator
- The final response is generated, converted into speech using a Text-to-Speech (TTS) service (using ElevenLabs), and streamed back to the user through LiveKit
- The system maintains state and context in a PostgreSQL database, storing chat history, agent traces, tool calls, confirmations, and logs for continuity and traceability
- Logging Service captures runtime events, pipeline activity, and errors (STT, TTS, tool calls), ensuring observability and debugging capabilities

Task-Based Orchestration: Instead of relying on large system prompts, the system uses JSON-based task definitions. Each task defines a specific goal, required inputs, and completion conditions. The orchestration engine dynamically selects tasks, manages workflow progression, and routes conversations to appropriate agents. All actions require explicit user confirmation, and external interactions are handled through centralized adapters.
Configurable AI Pipeline: A JSON configuration layer defines the providers used for STT (Deepgram), LLM (e.g., OpenAI or Gemini), and TTS (ElevenLabs). This abstraction allows easy switching of models and supports experimentation without changing core logic.
Real-Time Voice and Extensibility: LiveKit and LiveKit Agents provide real-time audio streaming. The system supports multi-agent workflows, agent handoffs, and features such as greeting, data extraction, and workflow execution. It also supports multilingual interaction (e.g., English/German), ensuring responses match the user's language.

System Workflow